

A Registry for Coastal-Simulation Meshes

What we want to build, why it's a data-engineering problem, and zero assumed background on meshes or coastal modeling.

First, a two-minute primer (skip if you already know)

What is a mesh?

A **mesh** is a digital quilt of small triangles that covers a piece of geography — for our purposes, an ocean basin, a bay, a stretch of coastline. Each triangle is a tiny patch where a physics simulator computes things like water level, current speed, and wave height. Coarse triangles in the deep ocean (where details don't matter much), fine triangles near the coast (where they do). The mesh is just the computational grid — it's what the math runs on.

Why does anyone need them?

When the news shows a forecast of how high storm surge will get during a hurricane, that forecast came out of a coastal-ocean simulator running on a mesh. The simulator we care about is called **ADCIRC**; it's open source, used by NOAA, the Army Corps of Engineers, academic labs, and consulting firms worldwide. **You can't run it without a mesh.** The mesh defines the geometry of the problem — without it the simulator has nothing to simulate.

So why is this a data problem?

Because building a high-quality mesh is hard, slow, and frequently redone. Researchers, agencies, and consultants have built thousands of useful meshes — covering everything from the entire Atlantic to a single fishing inlet. Almost all of them are valuable as test data, regression fixtures, baselines for new work, or starting points for derivative meshes. **And they're scattered everywhere with no common index.**

Imagine if every CSV dataset in the world lived in a different researcher's Dropbox folder — no Kaggle, no HuggingFace Datasets, no data.gov, no shared schema. That's the ADCIRC mesh world today.

The problem in detail

Mesh files exist in plain text (ASCII), with a known but messy format. There are five or six places people commonly publish them: an official ADCIRC examples page, a couple of GitHub repos, an academic group's S3 bucket, paper supplementary data, NOAA's operational archive. None of them talk to each other.

Pain points that a registry would fix

- **No discoverability.** Last week a teammate burned a whole session searching for a mesh that covers a specific region and includes a particular island — couldn't find one. There may be one out there, but no index lets you ask.
- **No classification.** No way to filter by mesh size ("under 10,000 triangles"), geographic scope ("Gulf of Mexico"), what physical features it represents (levees? breakwaters? open ocean?), or license ("public domain only").

- **No deduplication.** The same canonical mesh shows up in three different repos with no signal which is the authoritative copy or whether they've drifted.
- **No lineage for derivatives.** When someone takes a published mesh, refines it over a region of interest, or splices in new geographic features, that “modded” version has no link back to its parent. Future users can't tell what changed, or trust the provenance.
- **No license clarity.** Many fixtures show up with no stated license; downstream users don't know if they can redistribute, train models on, or commercially use them.

The proposed solution

Build a **federated mesh registry** — a community-driven catalog of meshes with consistent metadata, search, and a Python loader. Three components, each doing one thing well:

1. GitHub repo — the source of truth

A new public repo (separate from the simulation tools that consume it) holds a **manifest** — a structured file (TOML/JSON/YAML, TBD) with one entry per mesh: name, source URL, content hash, license, bounding box, size, what physical features it includes, **derived-from pointer**, contributor, and review state. Submissions are PRs against the manifest, validated by CI. The community already lives on GitHub, so the contribution path matches existing habits.

2. HuggingFace Datasets — the public mirror & frontend

CI publishes the manifest + meshes to a HuggingFace dataset on each release. We get all of this **for free**: a searchable, filterable web table (license badges, thumbnails, mesh size, region tags), dataset cards (citations, schema, examples), download counts (which gives us a “most popular” signal), CDN, search-engine indexing, and a `datasets.load_dataset(...)` Python loader other tools can pip-install and consume. The cost of building a custom website drops to near zero.

The catch: a mesh file isn't HuggingFace's native data model (it expects Parquet/Arrow columnar data). We'd publish meshes in raw-file mode with a custom loader script — a documented pattern, but it skips some of HF's nicer tooling. Optionally we publish a parsed-to-Parquet sidecar for queryability.

3. Python package — the programmatic interface

A thin Python package wraps the loader so downstream tools can do something like:

```
registry.find(
    bbox=(-75, 35, -70, 41),
    features=["levee", "open_ocean"],
    max_size=10_000,
    license="public-domain",
)
```

... and get back ready-to-use mesh objects. **Federated, not owned by any single project** — the mesh-generation library imports from it; doesn't host it.

Why this is interesting from a data-eng angle

- **Schema design from raw heterogeneous sources.** Reverse-engineer 6–8 existing mesh corpora into a single canonical schema that survives translation losses and stays queryable.
- **Federation patterns.** One source of truth (GitHub), multiple distribution surfaces (HuggingFace Datasets, optional Zenodo for DOIs, optional pip package). How do you keep them in sync without vendor lock-in? How do you handle deletions?
- **Provenance / lineage as a DAG.** Every derived mesh knows its parent and the operations applied (*refine_box(bbox)*, *add_island(ring)*, etc.). Effectively Git for meshes — though we don't need the full DVCS, just clean ancestry queries.
- **AI-assisted bundle generation (longer term).** Once meshes have feature vectors derived from the schema, an ML layer can answer queries like *“give me a balanced regression set: 5 small inlets, 3 estuaries, 2 ocean-scale, license-clean, feature-diverse, under 50K triangles total.”* Clustering, similarity search, or LLM-driven natural-language queries all fit. The schema is intentionally rich enough to feed any of these without future redesign.
- **Community contribution model.** PR-driven submission with automated schema validation in CI — classic data-eng ETL plus governance. Good test of how to design an extensible schema where contributors can add fields without breaking existing consumers.

Phasing

- **Phase 0 (now).** Research deliverable: survey existing registries, propose the schema, recommend the build path. No code — this is the design doc that justifies the new repo.
- **Phase 1 (~1–3 weeks).** Bootstrap the new repo. Schema implementation, manifest format pick, Python loader, PR-based contribution workflow with CI validation, ~5 seed meshes.
- **Phase 2.** HuggingFace publishing pipeline. CI generates the dataset card and pushes to HF on release. Public web frontend live.
- **Phase 3 (exploratory).** AI-assisted bundle generation. Pick one paradigm (clustering / similarity search / LLM-driven NL) and prototype.

Where I'd love your input

- Manifest format: TOML vs JSON vs YAML vs Parquet sidecar — what do you reach for when humans edit and machines validate?
- Storage: raw ASCII (round-trip-safe but not queryable) vs pre-parsed Parquet/Arrow (queryable but lossy without care)? Probably both — but how do you keep them consistent?
- Versioning: Git LFS vs DVC vs HuggingFace revisions vs all three? Each mesh file is 0.1–100 MB; the catalog might grow to 10K entries.
- Provenance graph storage: edges in the manifest vs separate lineage file vs render only in HuggingFace dataset cards?
- GitHub → HuggingFace sync: GitHub Actions on release tag, or continuous push? How do you handle contributor deletions / license retractions cleanly?

Reference links

- Tracking issue: github.com/domattoli/ADMESH/issues/6
- What ADCIRC is: adcirc.org
- Existing test suite (regression-focused, not a discovery registry):
github.com/adcirc/adcirc-testsuite
- HuggingFace Datasets: huggingface.co/docs/datasets